

Documento de Explicação da Lógica

Etapa 1	Apresentar o sistema e coletar os dados do usuário com validação de cada entrada
Etapa 2	Calcular diretamente o tempo de recarga e o custo com base na energia necessária
Etapa 3	Aplicar desconto para usuários Premium e somar a taxa de sessão

3. Início do Sistema e Coleta de Dados

O programa começa com uma apresentação visual no terminal, deixando claro para o usuário que o sistema foi iniciado:

```
=====
      Iniciando o sistema de recarga
=====
```

Em seguida, o sistema coleta cinco informações do usuário. A diferença importante em relação a uma abordagem simples é que cada entrada é validada antes de prosseguir — o programa não aceita dados inválidos e pede novamente com uma mensagem de erro clara.

3.1 Nome do usuário

A primeira entrada é o nome, sem validação específica — qualquer texto é aceito.

```
name = input("Digite seu nome: ")
```

3.2 Tipo de usuário (com validação)

O tipo de conta é coletado como número (1 para Padrão, 2 para Premium). Um laço while garante que o usuário só avance com uma opção válida:

```
while True:
    try:
        type = int(input("1 - Padrão / 2 - Premium: "))
        if type not in (1, 2):
            raise
        break
    except:
        # Exibe mensagem de erro e repete
```

3.3 Capacidade da bateria

A capacidade total da bateria é informada em kWh. O sistema rejeita valores negativos e entradas não numéricas.

3.4 Carga atual

O sistema valida que a carga atual não seja negativa e não ultrapasse a capacidade total da bateria. Isso evita situações impossíveis fisicamente.

3.5 Carga final desejada

A carga final deve ser maior que a carga atual (faz sentido: você quer carregar, não descarregar) e não pode superar a capacidade. Ambas as condições são verificadas.

4. Cálculo do Tempo e Custo — `simulate_recharge()`

Com os dados coletados, a função de simulação calcula tudo de forma direta e eficiente:

4.1 Energia necessária

A quantidade de energia que precisa ser transferida é simplesmente a diferença entre a carga final desejada e a carga atual:

```
total = final_load - current_load
```

4.2 Tempo estimado

Usando a potência do carregador (7,5 kW), o tempo é calculado diretamente por divisão:

```
potency = 7.5
minutes = total / potency
```

Por exemplo: se precisar de 30 kWh e o carregador entrega 7,5 kW, o tempo será $30 / 7,5 = 4$ horas.

4.3 Caso especial: bateria já no nível desejado

Se a carga atual já for igual à carga desejada (diferença zero), a função retorna tempo e custo zerados, evitando qualquer cálculo desnecessário.

```
if total != 0:
    # calcula normalmente
else:
    return 0, 0
```

5. Cálculo de Custo — `calculate_cost()`

O custo final é calculado em três passos:

5.1 Tarifa base

Cada kWh consumido custa R\$ 0,75:

```
cost = full_energy * 0.75
```

5.2 Desconto Premium

Usuários do tipo 2 (Premium) recebem 10% de desconto sobre o valor base:

```
if type == 2:
    descont = cost * 0.10
else:
    descont = 0
```

5.3 Taxa de sessão

Uma taxa fixa de R\$ 0,50 é somada ao final, independentemente do tipo de usuário ou da quantidade carregada:

```
cost_final = cost - descont + 0.50
```

6. Formatação do Tempo — format_minutes()

Uma função auxiliar converte o tempo decimal (em horas) para um formato legível em minutos e segundos:

```
def format_minutes(time):
    minutes = int(time)
    seconds = int((time - minutes) * 60)
    return f"{minutes}min{seconds}s"
```

Por exemplo, 1.5 horas vira "1min30s". Isso torna a saída muito mais próxima do que um sistema real de recarga mostraria ao usuário.

7. Relatório Final

Ao final, o programa exibe um relatório bem formatado com todas as informações da sessão:

```
=====
                        Relatório da recarga
=====
Nome: [nome do usuário]
Tipo: Padrão ou Premium
Tempo esperado de recarga: [Xmin Ys]
Total de energia consumida: [X.XX] kW
Custo: R$ [X.XX]
=====
                        Sistema de recarga Finalizado
=====
```

A abertura e o fechamento com linhas de separação dão ao programa uma aparência profissional, parecida com um terminal real de cobrança.

8. Conclusão

O projeto cumpre com sucesso todos os objetivos do Sprint 1: simula uma sessão de recarga, calcula o custo com regras diferenciadas por tipo de usuário e exibe um relatório formatado.

O destaque da implementação principal é a robustez na coleta de dados — o uso de laços while com try/except garante que o programa nunca travará por causa de uma entrada inválida, o que é essencial para qualquer sistema voltado ao usuário final.

O único ponto a corrigir antes de rodar é o erro de digitação na palavra "raise" dentro da validação do tipo de usuário. Com essa correção, o programa está pronto para uso.

Documento gerado com base no código-fonte do Sprint 1